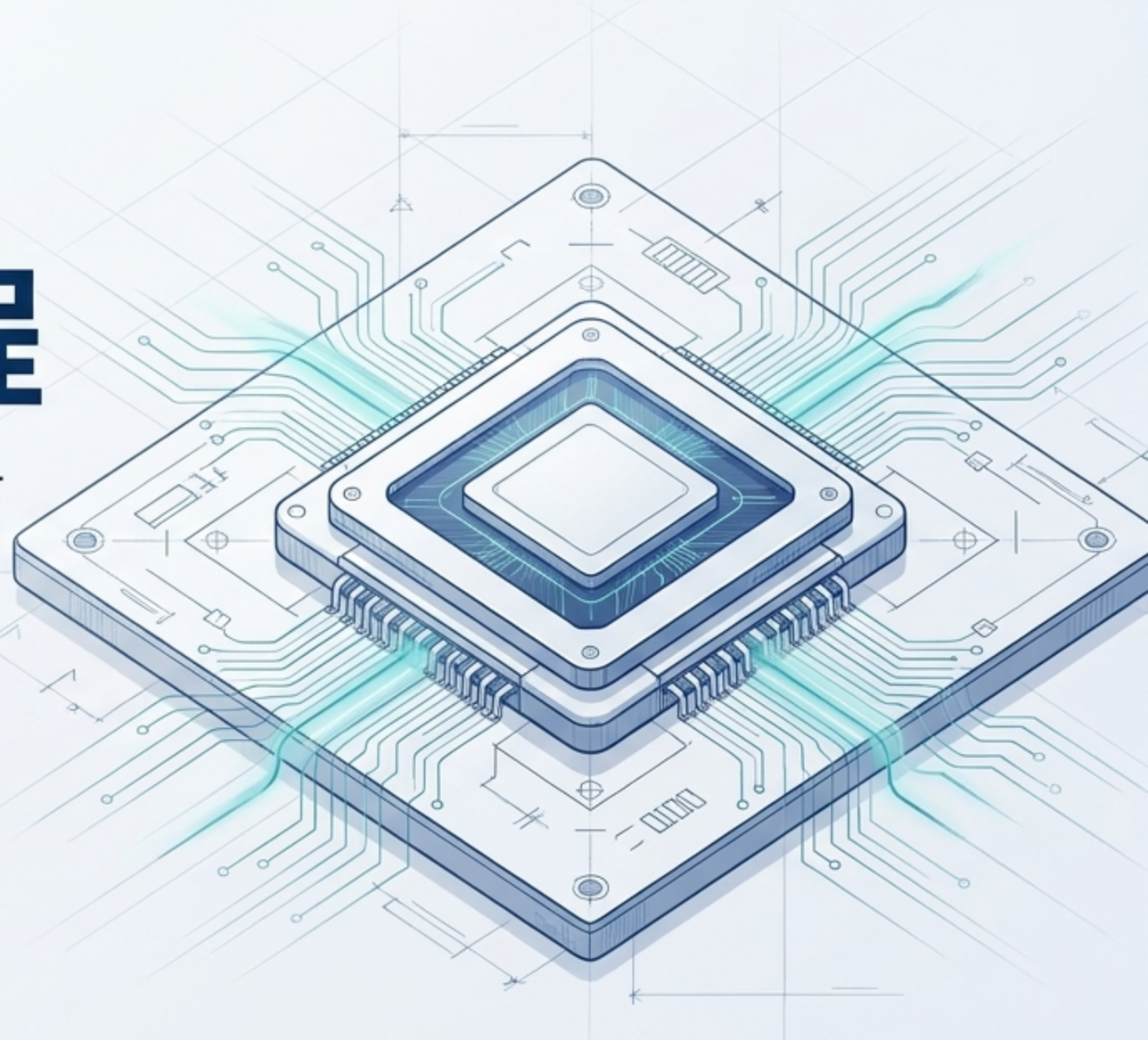
智能体驾驭工程

让大模型在真实世界落地的工程基石（Harness Engineering）



致网科技 × 模智空间 | 2024年度高级架构指南

Gemini Notebook

交互范式的三次跃迁：从被动响应到目标驱动

Chatbot · 聊天机器人

核心机制：被动式问答引擎

基于用户的单次提问进行针对性回答，交互结束即终止。模型无记忆、无状态，仅作为问答工具被动响应，不具备主动规划与行动能力。

你说一句，我回一句，单次交互即闭环，无后续自主行动。

Workflow · 智能工作流

核心机制：预设式流程编排

开发者预先设计完整的业务路径与逻辑，模型嵌入流程中作为特定节点。路径固定、边界清晰，无法处理超出预设范围的需求。

铁轨铺到哪，火车开到哪，严格遵循预设路径，缺乏灵活应变。

Agent · 自主智能体

核心机制：目标驱动的自主进化

以用户目标为核心，自主拆解任务步骤、动态选择工具、监控执行结果并实时调整策略，具备“感知-规划-行动”的自主闭环能力。

定好目标，自己想办法，规划路径、调用工具直到完成任务。

Gemini Notebook

核心重构：模型决定智商上限，Harness决定落地能力

Agent = Model + Harness

Model
智商上限：推理、决策

- 推理
- 决策

智商上限

Harness
工程底座：任务执行、状态管理、安全管控、异常处理

- 任务执行
- 状态管理
- 安全管控
- 异常处理

稳定　安全　高效

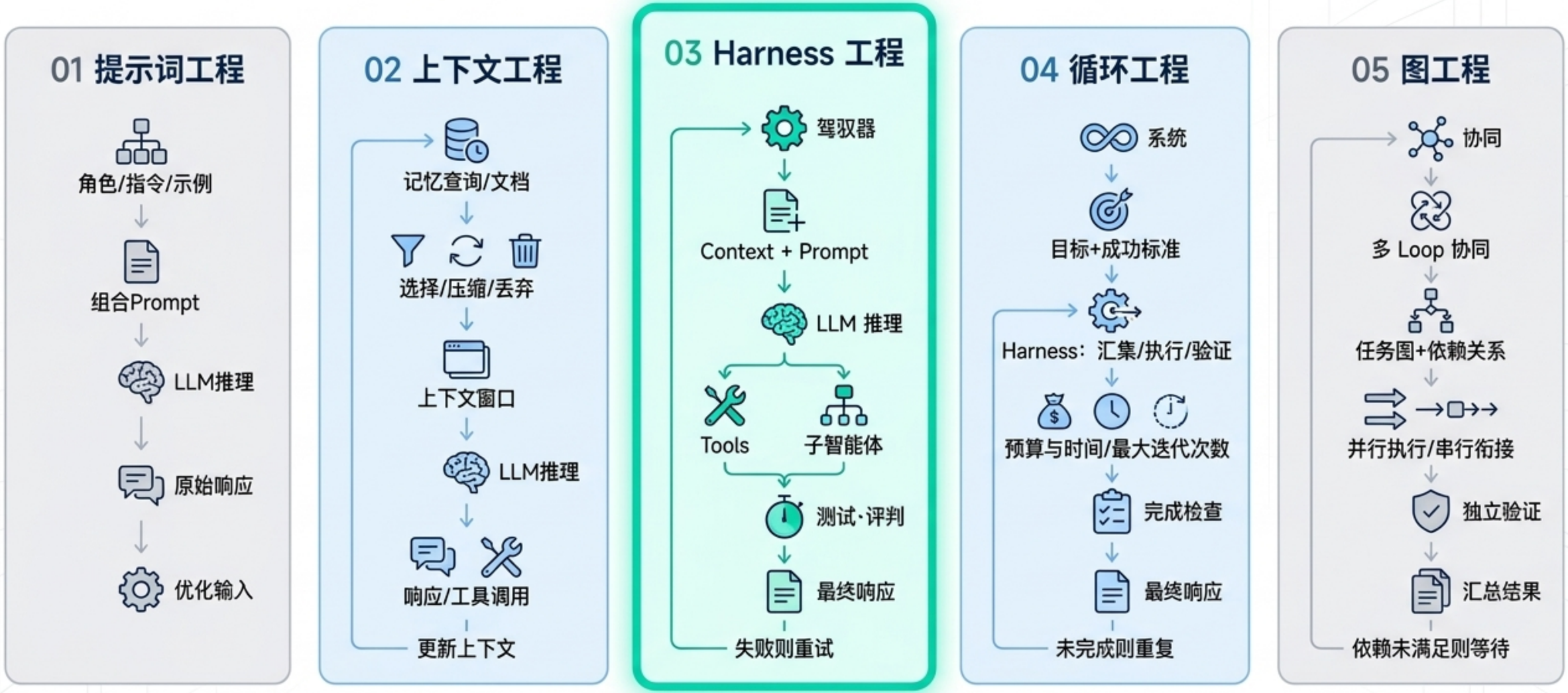
计算机（硬件）	LLM Agent（架构）
CPU	LLM
RAM	上下文窗口
硬盘	长期记忆
设备驱动	工具集成
操作系统	Agent Harness
应用程序	Agent

同一模型·只改 Harness　Terminal Bench 2.0　52.8% → 66.5%　+13.7%（+13.7 个百分点）

Gemini Notebook

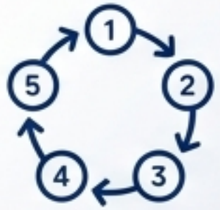
架构透视：AI 智能体五大工程层级依赖

行业技术迭代始终遵循外层包裹内层的逻辑。图工程依赖稳定的驾驭层，驾驭层依赖精准的上下文。Harness 是防止隐蔽故障的根基。



Gemini Notebook

组件解剖：生产级 Harness 的 11 个核心模块



1. 编排循环

ReAct / TAO · 循环执行



2. 工具系统

注册 · 校验 · 执行



3. 记忆系统

短时 · 长时 · 持久化



4. 上下文管理

压缩 · 屏蔽 · 按需加载



5. 提示词构建

分层组装 · 优先级



6. 输出结果解析

tool_calls · 结构化输出



7. 状态管理

记录 · 更新 · 断点恢复



8. 异常处理

重试 · 修复 · 熔断



9. 安全防护

输入 · 输出 · 工具



10. 自我验证

规则 · 可视化 · 模型裁判

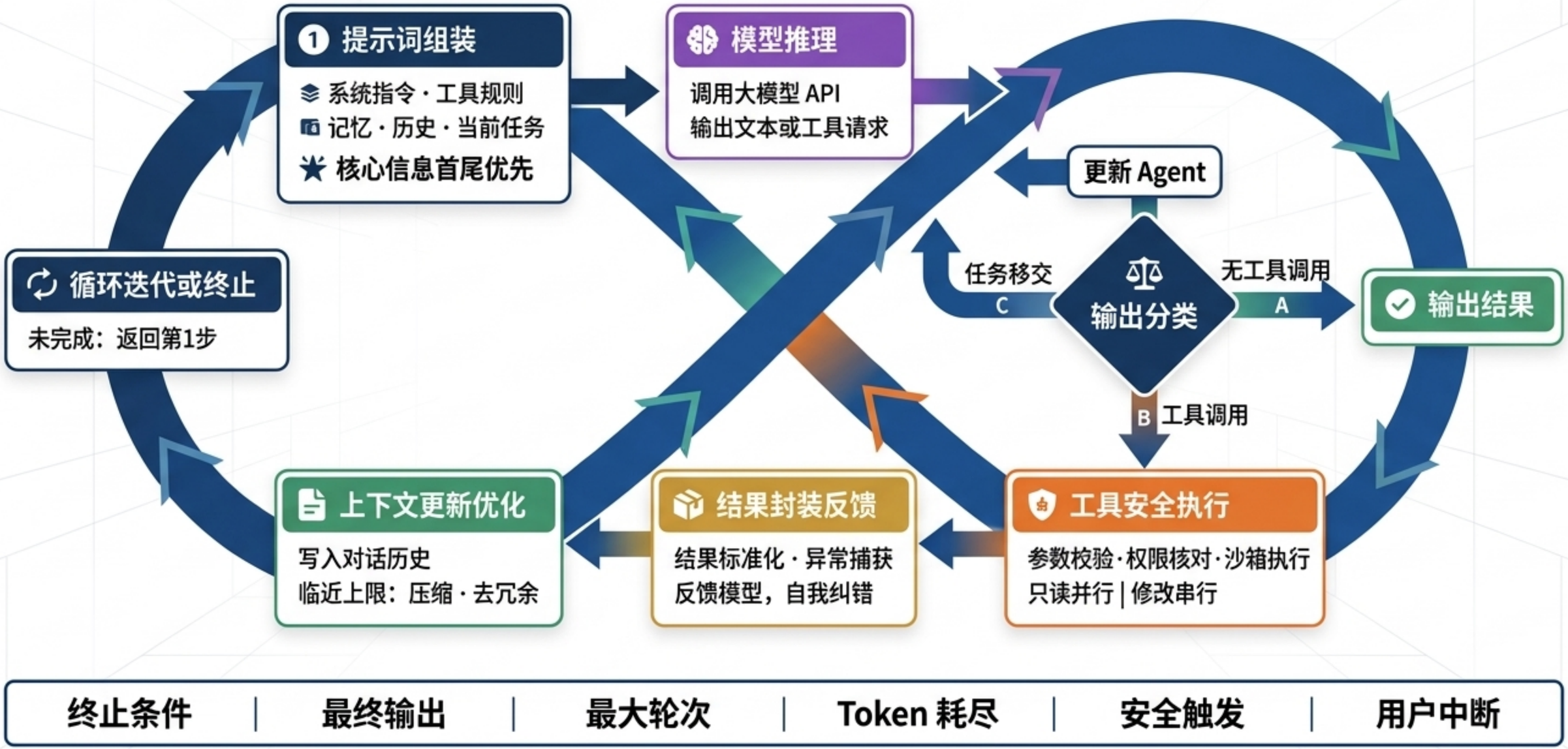


11. 子Agent编排

拆解 · 协同 · 汇总

Gemini Notebook

运转飞轮：Harness 完整生命周期与控制回路



终止条件	最终输出	最大轮次	Token 耗尽	安全触发	用户中断

Gemini Notebook

架构师的决策树：Harness 工程的七大核心选型

1 单 / 多 Agent

优先强化单 Agent（工具>10且功能重叠或任务领域彼此独立，再拆分）

多 Agent：调用开销 | 上下文损失

2 运行循环

ReAct（灵活度高 | 迭代成本高）
vs
规划执行
（速度更快 | 最高 3.6 倍）

3 上下文管理

清除　总结　屏蔽　笔记

保留推理轨迹，精简工具输出

Token ↓ 26%-54% | 准确率 >95%

4 自我验证

计算性验证（测试、Linters | 确定性依据）
推理性验证（LLM 裁判 | 识别语义问题 | 增加延迟）

5 安全权限

宽松模式（高效率 | 高风险）
vs
严格模式（高安全 | 低效率）

6 工具范围

最小可用 | 精简数量 | 按需加载

避免工具过多导致决策混乱

7 Harness 的厚度

核心决策交给模型 vs 仅保留基础能力

流程调度 | 安全防护 | 验证校验

Gemini Notebook

敏捷构建：6 步搭建生产级 Harness MVP



1 项目规范

技术栈 · 目录规范
测试指令 · 禁用行为



2 权限边界

默认只读 · 沙箱写入
自主 / 受限 / 高危
先严后松



3 本地校验

测试 · Linter · 代码检查
依赖管控 · 导入拦截
报错 -> 修复 -> 再测试



4 验证熔断

提交前自测
连续失败 -> 终止任务
定时巡检 · 防规范漂移



5 可观测性

行为日志 · 工具调用
Token 消耗
故障溯源 · 问题复盘

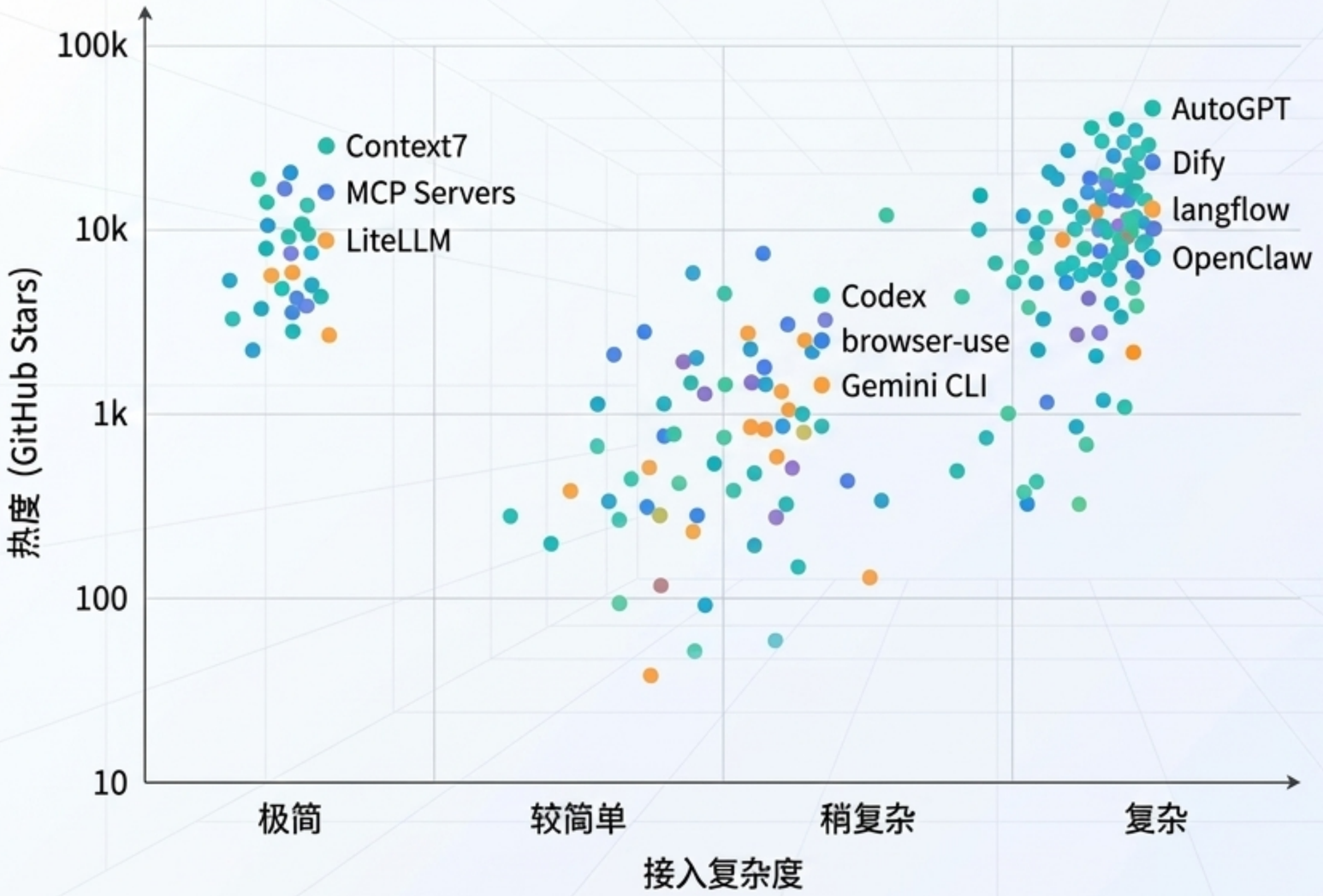


6 人工审核

生产数据 · 基础设施
安全配置 · 强制审批
效率 × 安全

Gemini Notebook

生态全景坐标：154个主流 Agent Harness 分布矩阵



高热度通用开源 Harness 框架全部集中在右侧高复杂度区间；

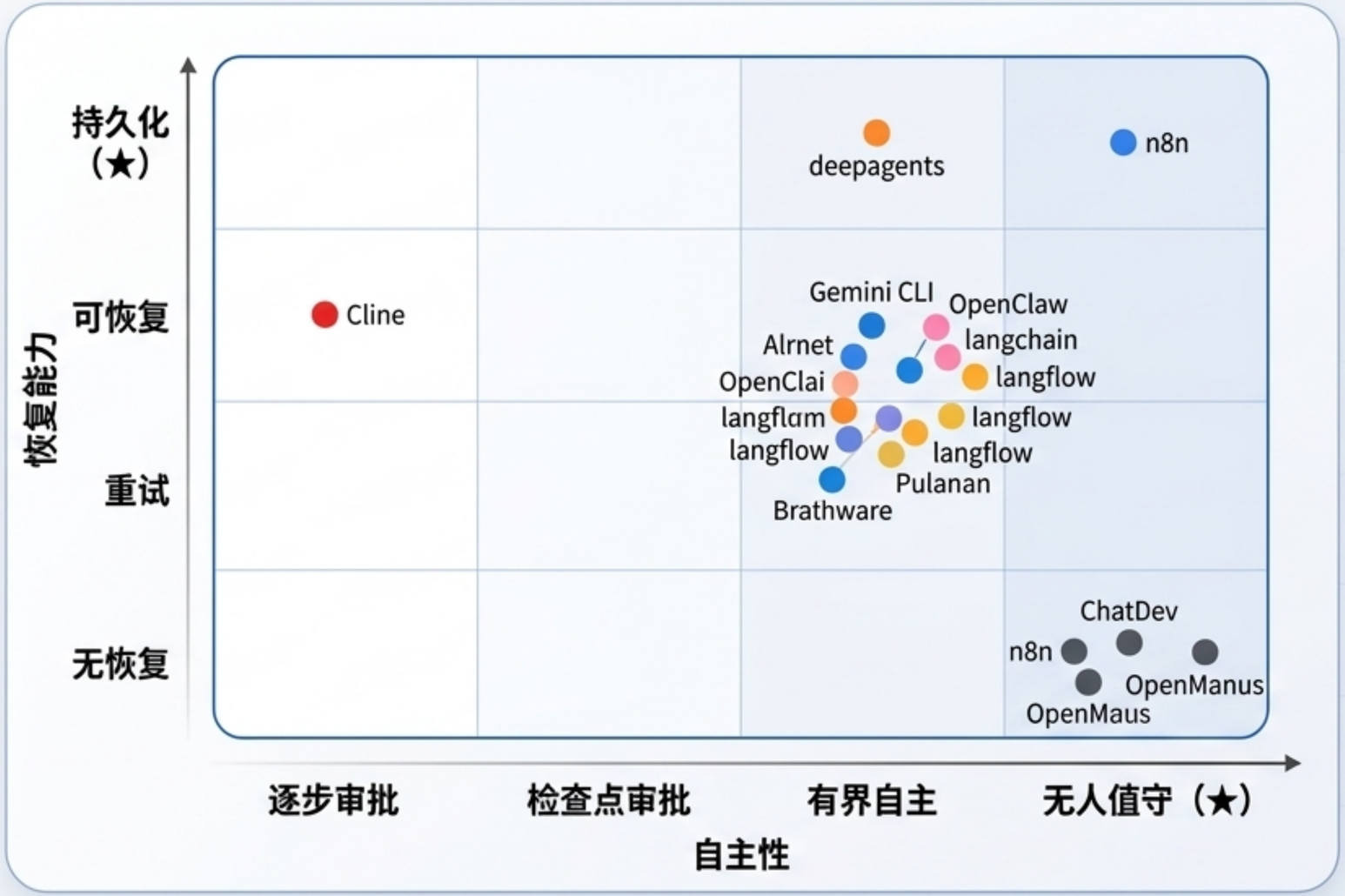
厂商配套 SDK、轻量化本地规则工具偏向左侧简易区间；

代码智能体、中型原型框架大多集中在中等杂度区域，完整多智能体协作平台均为高复杂度全栈架构。

Gemini Notebook

高可用试金石：自主权限与故障恢复能力矩阵

每个框架被设计赋予多大自主权，任务中途终止后如何恢复。★ 无人值守 = 可无人看管运行，★ 持久化 = 重启后仍可继续



- 中型生产级框架普遍落在「无人值守 + 断点恢复」区间；
- 追求极致安全的代码工具采用分步人工审批机制；
- 超长周期自动化平台同时具备最高自主权限与持久化状态存储；
- 简易原型框架大多缺少完整断点恢复能力。

Gemini Notebook

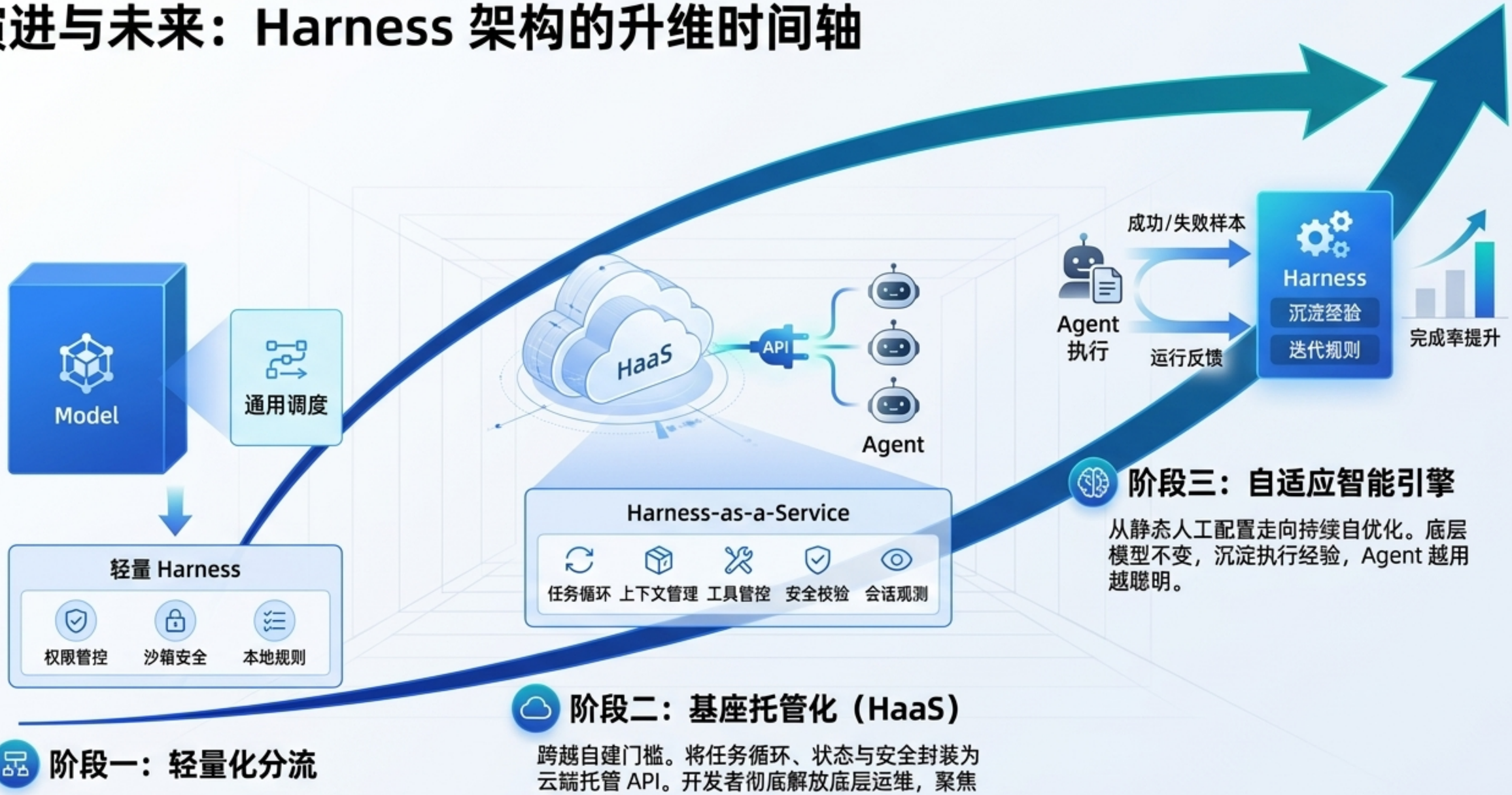
巅峰对决：标杆级 Harness 架构理念横评

Claude Code 与 OpenAI Codex 是两套标杆级 Harness 工程实现，内核秉持截然不同的架构理念。前者适配不可中断的长时间独立任务，后者更适合多人协作、需要规范复盘的规模化研发场景。

对比维度	Claude Code	Codex	核心取舍
控制平面	动态提示组装 实时注入、覆盖与裁剪 灵活轻量 | 追溯较弱	结构化指令片段 固定标记、类型与优先级 清晰可追溯 | 前期较重	动态生产流程 vs 标准化制度文件
会话连续性	状态集中于 queryLoop 循环内就地处理异常 反应快 | 崩溃后需重建	Turn 事件独立归档 会话 ID 与历史持久化 恢复强 | 处理链路更重	脑内待办 vs 完整日志
本地规则治理	CLAUDE.md + Skills Hooks + Session Memory 落地快 | 规则易碎片化	结构化控制平面 版本、灰度、分发与审计 规模治理强 | 上手成本高	工位便签 vs 标准流程

Gemini Notebook

演进与未来：Harness 架构的升维时间轴



阶段一：轻量化分流

通用能力内化进模型，Harness 剥离变薄，专注差异化与合规治理。

阶段二：基座托管化（HaaS）

跨越自建门槛。将任务循环、状态与安全封装为云端托管 API。开发者彻底解放底层运维，聚焦业务逻辑。

阶段三：自适应智能引擎

从静态人工配置走向持续自优化。底层模型不变，沉淀执行经验，Agent 越用越聪明。

Gemini Notebook